

UNIVERSIDAD PERUANA UNIÓN • FACULTAD DE INGENIERÍA Y ARQUITECTURA

TRABAJO AUTÓNOMO: SESIÓN 2

Operaciones Asíncronas y Flujos de Datos con Corrutinas y Flow en KMP

Modalidad Autónoma • Pregrado • VIII Ciclo • Semestre 2026-2 • Asignatura: Desarrollo de Aplicaciones Móviles

PROPÓSITO & MODALIDAD DE TRABAJO

Objetivo de la Actividad

- > Consolidar de manera independiente los conocimientos de modelado y programación asíncrona en **commonMain**.
- > Ampliar el dominio de PharmaMobile con operaciones suspend y flujos Flow.
- > Documentar evidencias y registrar incrementos mediante Git.

Enfoque Metodológico

Trabajo estrictamente individual que refuerza la autonomía técnica, la trazabilidad del código y la preparación para arquitecturas reactivas y offline-first.

PASO 1: RECUPERACIÓN & CONTROL GIT INICIAL

Verificación de Estado

- > Apertura del proyecto PharmaMobile con los modelos del Reto 01 ya implementados.
- > Comprobación obligatoria del estado del repositorio local antes de iniciar modificaciones.
- > Garantía de un workspace limpio y sincronizado.

Comando de Validación

```
cd C:\dev\PharmaMobile  
git status
```

Confirma que no existan conflictos ni cambios sin registrar antes de aunar código asíncrono.

PASOS 2 & 3: DEPENDENCIAS Y DATOS SIMULADOS

Soporte de Corrutinas

- > Verificación de la librería `kotlinx-coroutines-core` dentro del módulo compartido `commonMain`.
- > Creación de una fuente temporal de productos simulados para pruebas de dominio.

```
private val productosSimulados = listOf(  
    Producto(1, "Paracetamol", 8.50, 100),  
    Producto(2, "Ibuprofeno", 12.00, 50),  
    Producto(3, "Amoxicilina", 18.50, 20)  
)
```


PASOS 4 & 5: OPERACIONES ASÍNCRONAS (SUSPEND)

Simulación de Latencia

- > Uso de la palabra clave **suspend** para pausar y reanudar sin bloquear hilos.
- > La función `delay(1000)` emula una llamada a red remota o lectura de base de datos local.

```
suspend fun obtenerProductos(): List {  
    delay(1000) // Simula espera de red  
    return productosSimulados  
}
```


PASOS 6 A 8: MANEJO DE RESULTADOS (SEALED CLASS)

Jerarquía de Estados

- > Modelado de los estados de una operación asíncrona: **Cargando**, **Exito** y **Error**.
- > Evaluación exhaustiva de cada rama mediante la expresión **when**.

```
sealed class ResultadoProductos {  
    data object Cargando : ResultadoProductos()  
    data class Exito(val list: List) : ResultadoProductos()  
    data class Error(val msg: String) : ResultadoProductos()  
}
```


PASOS 9 A 11: INTRODUCCIÓN A FLOW (EMIT Y COLLECT)

Secuencias Asíncronas Múltiples

- > Mientras `suspend` retorna un único valor, **Flow** emite una secuencia de valores a lo largo del tiempo.
- > **emit()** publica nuevos datos; **collect()** los consume de forma reactiva.

```
fun observarEstados(): Flow = flow {  
    emit("Iniciando")  
    delay(1000)  
    emit("Finalizado")  
}
```


PASOS 12 A 14: FLOW DE PRODUCTOS & COPY()

Emisión de Cambios de Inventario

- > Emisión inicial de lista vacía o base, seguida tras un retraso de la lista de productos simulados.
- > Uso de `copy()` para simular actualizaciones dinámicas en el stock del inventario.

```
fun observarProductos(): Flow<> = flow {  
    emit(emptyList())  
    delay(1000)  
    emit(productosSimulados)  
}
```


PASOS 15 A 17: FLUJO INTEGRADO DE ESTADOS

Composición Avanzada

- > Combinación de **Flow** con Sealed Classes para emitir secuencias de estados (Cargando → Éxito).
- > Prepara el puente directo hacia arquitecturas MVVM y StateFlow en la UI.

```
fun cargarProductos(): Flow = flow {  
    emit(ResultadoProductos.Cargando)  
    delay(1000)  
    emit(ResultadoProductos.Exito(productosSimulados))  
}
```


PASOS 18 & 19: TABLA COMPARATIVA (SUSPEND VS FLOW)

Característica Técnica	Función suspend	Flujo Flow
Naturaleza del resultado	Retorna un único valor al finalizar.	Emite múltiples valores a lo largo del tiempo.
Mecanismo de consumo	Llamada directa asíncrona.	Recolección mediante collect { }.
Caso de uso principal en PharmaMobile	Consultas puntuales o inserción de un pedido.	Observar cambios de stock, inventario o lista de pedidos en tiempo real.

PASOS 20 & 21: ARQUITECTURA & EVIDENCIAS

Arquitectura Futura

Evolución desde datos simulados actuales hacia Repositorios, Ktor REST API, Room/SQLDelight y Compose UI.

Evidencias de Ejecución

Capturas o salidas por consola comprobando la correcta emisión y recolección de los estados asíncronos.

Informe en PDF

Consolidación del reporte individual nombrado bajo la estructura `Apellido_Nombre_Sesion02_Reto02.pdf`.

PASO 22: VERSIONADO GIT & CIERRE AUTÓNOMO

Comandos de Registro Final

- > Verificación de cambios limpios mediante `git status`.
- > Preparación del staging area con `git add` .
- > Commit semántico descriptivo: **feat: agregar corrutinas y flujo de productos.**

CORE

Dominio compartido asíncrono y reactivo listo para
Compose UI.